- **Performance**: We don't *always* necessarily have to search for the optimal solution. We can sometimes settle for a sub-optimal implementation if that's efficient enough and performance is not critical.
- There are times, however, when we need a better algorithm to scale out to large datasets: as a reference, consider that any algorithm belonging to `O(n2)` (or higher classes) can't be *scaled* indefinitely, i.e. it can't be applied to an arbitrarily large input without becoming unpractical.
- **The importance of analysis**: Sometimes even problems that seems trivial to solve have room for impactful improvement and need careful analysis. Keeping track of the portioning of a set of elements into disjoint subset is one of those cases: probably we could settle for the naïve linear time solution, but it is so fundamental as part of many graph algorithms that we just need to optimize it as much as possible.
- Speaking of which, sometimes you don’t know how much something can be optimized, or if you have found the best possible solution. This is the case for disjoint-set: we do know a theoretical lower bound for the running time of operations of a disjoint set, but we don’t know if there exists an algorithm that runs with that bound, or even just any other algorithm faster than the ones we know.
- **Ackerman function**: You should now be familiar with the *Ackerman function* and its inverse: the first one grows incredibly fast, the second one incredibly slowly. The *inverse Ackerman function*, whose value won’t be greater than 5 for any integer that could fit on a computer, models the order of magnitude of the